

One Bitstream, Many Decoders: A Portable, Minimal-Dependency Lossless Codec for Real-Time DICOM Viewing in Web-Based PACS

Kuldeep Singh

Abstract—Web-based Picture Archiving and Communication Systems (PACS) viewers must decode Digital Imaging and Communications in Medicine (DICOM) studies at interactive rates (about 25–60 frames per second for scrolling and cine playback), losslessly at the scanner’s native 10–16-bit depth, and in whatever execution environment the clinical endpoint provides: a native workstation, a server-side rendering gateway, a Node.js service, or a browser tab where only JavaScript and WebAssembly are available. This paper presents MIC (Medical Image Codec), a lossless pipeline of spatial prediction, 16-bit run-length encoding, and large-alphabet Finite State Entropy coding, together with a family of interoperable decoder implementations that all consume the identical compressed bitstream: pure Go, C with platform SIMD, multi-state instruction-parallel variants, a strip-parallel mode, a dependency-free ~20 KB JavaScript decoder, and three WebAssembly builds (Go, C, and C with POSIX threads). On a 39-image, eleven-modality benchmark corpus MIC reaches a geometric-mean compression ratio of $3.36\times$ (within about 2% of High-Throughput JPEG 2000, about 88% of JPEG-LS) and is the fastest of four in-process decoders on all 39 images on ARM64 and 36 of 39 on AMD64. In the browser, the same 7.18 MB radiograph decodes in about 140 ms with the pure JavaScript decoder, 17 ms with the 20 KB C-to-WebAssembly build, and 4 ms with the threaded WebAssembly build, versus about 125 ms and 70 ms for WebAssembly builds of HTJ2K and JPEG-LS decoding their own files. A time-to-display model over eight network profiles shows when decoder choice dominates (fast networks) and when compression ratio dominates (slow networks). All live browser decodes are verified bit-exact against the source pixels.

Index Terms—medical image compression, lossless compression, PACS, DICOM, web viewer, WebAssembly, browser decoding, deployment portability, entropy coding, asymmetric numeral systems (ANS), finite state entropy (FSE).

I. INTRODUCTION

WEB-BASED Picture Archiving and Communication Systems (PACS) viewers have become a primary path for clinical image review: a radiologist opens a study in the browser, scrolls a multi-slice computed tomography (CT) series or a digital breast tomosynthesis (DBT) stack, and expects each frame to appear without a perceptible stall. Diagnostic review requires that the reconstructed image match the scanner output bit for bit, so the codec must be lossless at the native 10–16-bit depth, and the practical viewing bottleneck, based on experience building clinical PACS viewer software, is sustaining roughly **25–60 frames per second (FPS)** of decoded pixels for smooth scrolling and cine playback. A single tomosynthesis view can

comprise 60 or more slices and over 600 MB uncompressed, and a full screening study can exceed 2 GB, so compression is required for transfer; interactive display further requires that decompression be fast enough on the client that frame rate, not file size alone, is the binding constraint.

This paper starts from a second constraint that archival codec comparisons rarely make explicit: **the decoder must run wherever the viewer runs**. A modern imaging deployment spans several execution environments at once. A rendering gateway or archive node runs native code and benefits from Single Instruction, Multiple Data (SIMD) instructions and threads. A zero-install web viewer runs whatever a browser tab provides, which at the time of writing means JavaScript (JS) and WebAssembly (WASM), with threading available only under specific cross-origin isolation headers. A Node.js tier sits in between. Institutional software policy adds its own limits: a hospital web viewer cannot ask users to install native codec plugins, and security review is far simpler for a small dependency-free artifact than for a large compiled library. A codec intended for real-time web PACS viewing therefore has three coupled requirements: **high decode performance**, **minimal dependencies**, and **portability across deployment environments**, all without sacrificing lossless 16-bit reconstruction.

Digital Imaging and Communications in Medicine (DICOM) [1] already supports several lossless formats, including JPEG 2000 [2], High-Throughput JPEG 2000 (HTJ2K) [3], JPEG-LS [5] (a lossless/near-lossless predictive JPEG standard), and a basic run-length scheme. Each makes a different trade-off among ratio, decode speed, and implementation complexity [26], [27]. JPEG-LS usually produces the smallest files; HTJ2K improves substantially on classical JPEG 2000 decode speed. Neither was designed around the joint constraint this paper targets, and their browser paths are WebAssembly ports of large native libraries rather than purpose-built client-side decoders. Server-side transcoding can paper over format gaps, but it adds latency, cost, and an extra failure path.

A. Target workflow: client-side multi-frame viewing

The viewer path comprises three stages that must stay within the end-to-end frame budget:

- 1) **Fetch** a compressed frame (or strip-parallel blob) from object storage or a PACS gateway into the client.
- 2) **Decode** the frame to native-depth pixels on the client (native code, JavaScript, or WebAssembly), without a server re-encoding step.

- 3) **Display** the frame in a canvas or WebGL viewport while the user scrolls or plays cine.

Under that model the codec succeeds only if decode time per frame leaves enough budget for fetch and display at 25–60 FPS on the environment actually available at the endpoint. Large mammography and tomosynthesis frames are the stress cases; small magnetic resonance (MR) and positron emission tomography (PET) frames are latency-sensitive for series scrubbing. Section VIII reports in-browser measurements against this target for every decoder variant; Section VII establishes the ratio and native throughput context.

B. Why 16-bit-native coding matters for that workflow

Medical pixels are commonly 10–16 bits wide, not the 8-bit samples assumed by most general-purpose entropy coders (Huffman, arithmetic coding, the Lempel-Ziv 1977 (LZ77) family, and the Finite State Entropy / asymmetric numeral systems stage inside Zstandard (Zstd) [18]). When an 8-bit coder is applied to 16-bit residuals, it has two poor options:

- 1) **Split each residual into a high byte and a low byte** and code them separately. This doubles the number of coding steps and discards the relationship between the two bytes.
- 2) **Cap the alphabet** and store rarer values as raw data, which wastes space.

In contrast to codecs that split 16-bit pixels into byte pairs, a 16-bit-native design treats each predicted residual as a single 16-bit symbol, so the entropy model can capture correlations between the high and low byte directly. For example, if the predicted value is 1020 and the true pixel is 1023, the residual is +3. Residuals near zero dominate smooth tissue; the high byte of a small signed residual is nearly deterministic given the low byte, yet a byte-oriented coder still spends bits on it. On the test images the mutual information between the two bytes ranges from 0.3 to 1.1 bits per pixel, which is an upper bound on the waste of byte-splitting. Section VII reports that MIC produces files 5–22% smaller than Delta+Zstandard coding the same residuals after byte-splitting; that measured advantage falls within the mutual-information ceiling and is consistent with the shared-information argument as the main source of the gain. For a web viewer, smaller files also reduce fetch time, so the same design choice that improves ratio helps the end-to-end frame budget.

C. What MIC is

MIC (Medical Image Codec) is a three-stage lossless pipeline built for high decode throughput on medical residuals: (i) a simple spatial predictor (average of the top and left neighbors), (ii) a 16-bit run-length encoder, and (iii) a Finite State Entropy (FSE) coder, a table-driven form of asymmetric numeral systems (ANS), extended to large alphabets (up to 65,535 symbols, with one code reserved as an overflow delimiter). Two decode-side parallel mechanisms complete the design: a multi-state decoder that interleaves several independent FSE chains to exploit instruction-level parallelism (ILP), and an optional strip-parallel mode called Parallel Image Compressed

Strips (PICS) that splits an image into N horizontal strips, each with its own FSE table and independently decodable bitstream, so a worker pool can decompress strips concurrently.

The distinguishing engineering property, and the focus of this paper, is that this one compressed format is served by a **family of interoperable decoder implementations** spanning the deployment spectrum: a pure Go codec with no external dependencies, a single-file C decoder with platform SIMD, a dependency-free ~20 KB JavaScript decoder, and three WebAssembly builds (compiled from Go, from C, and from C with POSIX threads for parallel strip decode). Every implementation consumes the identical bytes and reproduces the identical pixels, verified by checksums in the evaluation harness, so the choice among them is purely a deployment decision.

D. Contributions

- A **viewer-oriented problem framing**: lossless compression judged by whether client-side decode sustains interactive multi-frame rates in a web PACS across realistic deployment environments, not by archival ratio alone (Sections I-A, V).
- A **16-bit-native pipeline** (Delta + 16-bit RLE + extended FSE) that outperforms Delta+Zstandard on 34 of 39 images (about 10% smaller in geometric mean) while remaining simple enough for a small browser decoder (Section III).
- A **multi-state and strip-parallel decode path** (Section IV) that keeps single-threaded and multi-threaded throughput above cine targets on heavy studies, including tomosynthesis-sized frames.
- A **systematic cross-deployment evaluation** (Sections V, VIII): twelve decoder configurations measured live in a real browser on identical bytes (pure JavaScript, Go-to-WASM, C-to-WASM, threaded C-to-WASM, Web Worker pools, and WebAssembly builds of HTJ2K and JPEG-LS), each decode verified bit-exact, plus native and Node.js measurements on two architectures.
- An **end-to-end time-to-display model** over eight network profiles (Section VIII-D) that identifies when decoder choice dominates the user-visible latency and when compression ratio does.

a) *Why this combination is the contribution*: Spatial prediction, run-length coding, and ANS/FSE are each well established, so the claim is not inventing those stages. The contribution is that pairing a 16-bit-native entropy stage with multi-state parallel ANS decoding, and shipping that path as a set of small, dependency-free decoders across every tier of a web PACS deployment, removes a specific bottleneck: sustained real-time, browser-based decoding of DICOM images at native 16-bit depth. JPEG-LS's Golomb-Rice stage and HTJ2K's block coder retain per-symbol data-dependent branches that limit decode throughput; Zstandard's FSE is fixed at an 8-bit-oriented, 4,096-symbol alphabet. None of those pipelines simultaneously targets native 16-bit alphabet width, multi-state interleaved decode, and purpose-built deployment artifacts for interactive PACS viewing.

b) *Evaluation scope and statistical framing:* Claims in this paper are about **decoder throughput, compression ratio, and deployment feasibility on a deliberately diverse modality set**, following the systems-benchmark tradition (for example, Kodak’s 24-image suite and CLIC validation sets of roughly 41–61 images), not clinical validation of diagnostic interchangeability on a population archive. Clinical-validation studies of lossless codecs often use thousands of images [27]; the 39-image, eleven-modality corpus here, plus seven multi-frame cine studies, is chosen for breadth of pixel statistics and viewer stress cases, not for population inference. Larger multi-hospital validation is left to future work (Section IX).

c) *Scope:* Primary results cover lossless grayscale decode for single-frame and multi-frame viewing paths. Color whole-slide imaging, lossy modes, and progressive decoding are out of scope.

II. RELATED WORK

The DICOM lossless codecs. JPEG 2000 [2] uses a wavelet transform plus a sophisticated block coder (Embedded Block Coding with Optimized Truncation, EBCOT): strong ratios, but slow decoding. HTJ2K [3], [4] replaces the block coder with a faster one and decodes considerably quicker. JPEG-LS [5] uses a small predictor (the Median Edge Detector, MED) and Golomb-Rice coding; it is a widely deployed standard and a strong ratio baseline. The basic DICOM run-length scheme compresses little because it works on raw bytes with no prediction step. Research codecs such as the Context-based Adaptive Lossless Image Codec (CALIC) [6] and the Free Lossless Image Format (FLIF) [7] compress well on natural images but are not DICOM standards and lack browser decoders.

Web PACS viewer infrastructure. Browser-based DICOM viewing is an established clinical practice built on the DICOMweb family of web services [8] and open-source viewer frameworks such as the Open Health Imaging Foundation (OHIF) viewer [9], which decode pixel data client-side using WebAssembly [10] ports of native codec libraries compiled with Emscripten [11]. This paper follows that deployment model and uses the same vendored WebAssembly builds of OpenJPH (HTJ2K) [28] and CharLS (JPEG-LS) [29] that production web viewers employ as live, pixel-verified baselines, so the browser comparison reflects the paths a clinical web viewer would actually take at the time of writing.

JPEG XL. JPEG XL [12] is a general-purpose image format whose modular lossless mode also handles high-bit-depth data, and its absence from the live browser comparison deserves an explicit explanation rather than a silent omission. In informal cross-checks using the reference `libjxl` encoder and decoder called in-process (effort level 7; the harness is included in the accompanying code), JPEG XL reaches a better compression ratio than MIC on most of the same images, while MIC encodes and decodes markedly faster. The more important reason JPEG XL is not a live baseline here is that it is not a feasible option for the browser-viewer workflow this paper targets: the only mature browser-side JPEG XL decoder we identified at the time of writing, `@jsquash/jxl`, returns

an 8-bit RGBA `ImageData` object and so cannot represent lossless 16-bit grayscale medical pixels. JPEG XL therefore appears in the browser tables as an informational row: its real compressed file sizes (produced and round-trip-verified natively) are reported, but its decode rate is a native reference measurement, not a browser decode. Newer learned lossless codecs, such as L3C [13] and SReC [14], can reach still better ratios, but depend on neural-network runtimes and decode far more slowly, which makes them impractical for real-time web viewing; they are excluded from the quantitative comparison for the same reason.

Entropy coding background. Asymmetric Numeral Systems (ANS) [15], [16] is a family of entropy coders that replaces arithmetic coding’s interval math with simple integer state updates. FSE is a fast, table-driven version of ANS [17] made popular by Zstandard [18]. Coding directly over a large, multi-symbol alphabet rather than a byte stream is not itself new; for example, Mahapatra and Singh built a multi-alphabet arithmetic coder in FPGA hardware [19]. The limitation is that the widely deployed software implementations are tuned for byte streams (Zstandard caps FSE at 4,096 symbols).

Parallel ANS. Prior work has shown that ANS decoders run faster when several independent streams are decoded at once (Giesen on interleaved coders [20]; later work scaling this to many streams [21] and to GPUs [22]). MIC does not claim to invent ANS or interleaving; it applies these known ideas inside a 16-bit-native medical codec with a specific large-alphabet design, and then carries the same interleaved format across every deployment tier down to a browser worker pool.

Table I places MIC against the prior work on axes that matter for a web PACS viewer: lossless medical support, 16-bit residual coding, parallel decode, and browser deployability.

TABLE I
FEATURE COMPARISON OF MIC WITH RELATED CODECS AND ENTROPY CODERS FOR THE VIEWER-RELEVANT AXES. “16-BIT RESIDUAL AS ONE SYMBOL” CONCERNS THE ENTROPY STAGE ONLY: JPEG-LS, HTJ2K, AND JPEG XL HANDLE HIGH-BIT-DEPTH IMAGES, BUT THEIR ENTROPY STAGES DO NOT TREAT THE PREDICTED RESIDUAL AS A SINGLE 16-BIT SYMBOL IN THE SENSE USED HERE. “16-BIT LOSSLESS IN BROWSER” MEANS A PRODUCTION-READY PATH THAT RECONSTRUCTS NATIVE-DEPTH MEDICAL PIXELS CLIENT-SIDE AT THE TIME OF WRITING. SOURCE-LINE COUNTS ARE ORDER-OF-MAGNITUDE CONTEXT.

Feature	JPEG-LS	HTJ2K	JPEG XL	Zstd	rANS	MIC
Lossless medical	Yes	Yes	General	General	General	Yes
16-bit residual as one symbol	No	No	No	No	Configurable	Yes
Large alphabet (symbol cap)	N/A	N/A	N/A	4,096	Configurable	65,535
Adaptive table sizing	No	No	Yes	No	No	Yes
Multi-state ILP	No	No	No	No	Yes	Yes
Strip-parallel decode	No	Yes	Yes	Yes	Yes	Yes
16-bit lossless in browser	Via WASM	Via WASM	No	Partial	No	Purpose-built JS
Source lines	5k	20k	Large	N/A	N/A	1.1k

III. CODEC DESIGN

The pipeline is three stages, each simple enough to decode in one straight-line pass with very few branches: raw 16-bit pixels go through spatial prediction (subtract a guess based on neighbors), then 16-bit run-length encoding (RLE; collapse repeated values), then FSE entropy coding (remove the remaining redundancy), producing compressed bytes. The stage inventory is deliberately small because every stage must

be reimplemented faithfully in each deployment target (Go, C, JavaScript, WebAssembly), and a small specification is what keeps those implementations byte-compatible and auditable.

A. Spatial prediction

For each pixel, MIC predicts its value as the **average of the pixel above and the pixel to the left**, then stores only the difference (the residual). Pixels on the top edge or left edge use whatever single neighbor is available; the very first pixel is stored as-is. Division rounds down, and the decoder repeats the exact same arithmetic so reconstruction is exact.

This predictor was chosen because it is inexpensive, has almost no branches on the decode side, and works well on medical images.

a) Other predictors we evaluated: We compared four predictors (left-only, average, Paeth from PNG [24], and MED from JPEG-LS [5]) through the full pipeline. MED gave the best ratio (about 2.4% better than average) but decoded 1.5–2× slower because of its three-way branch and a diagonal dependency that blocks the fast branch-free loop. The average predictor was preferred for its consistency and speed, so it is the default. Table II summarizes the comparison.

TABLE II
PREDICTOR ABLATION SUMMARY: GEOMETRIC MEANS AND WIN COUNTS ACROSS ALL 39 IMAGES.

Predictor	Geo. Mean	Wins	Avg→X
Left-only	3.22×	2/39	−4.4%
Average (MIC)	3.36×	11/39	baseline
Paeth	3.39×	0/39	+0.8%
MED (JPEG-LS)	3.44×	26/39	+2.4%

B. Handling large jumps: overflow coding

Most residuals are small, but occasionally a pixel jumps far from its prediction. Rather than widen the whole stream to 32 bits to handle rare large values, MIC reserves one special code as a **delimiter**. For an image of effective bit depth d , the threshold is $\text{deltaThreshold} = 2^{d-1} - 1$ and the delimiter is $D = 2^d - 1$. A residual r with $|r|$ strictly less than the threshold (the comparison is strict $<$, not $\leq$) is stored directly as the single code $\text{deltaThreshold} + r$; in particular, a residual of zero maps exactly to deltaThreshold . A residual at or beyond the threshold is stored as the delimiter D followed by the raw pixel value.

On decode, the reader examines each code c : if $c \neq D$, the residual is recovered as $c - \text{deltaThreshold}$ and added to the prediction; if $c = D$, the next value is read as a raw pixel. The ranges are designed not to overlap, so there is never any ambiguity. In normal 8–16-bit clinical data the overflow path is rare; it exists to guarantee correctness, not for the common case. Table III shows the mapping for a representative bit depth.

C. 16-bit run-length encoding

The residual stream is turned into two kinds of runs: **same runs** (a count plus one repeated value, used when a value

TABLE III
OVERFLOW CODING EXAMPLE FOR $d = 12$ BITS ($\text{DELTA_THRESHOLD} = 2047$, $\text{DELIMITER } D = 4095$).

Residual r	Emitted code(s)	Notes
−2	2045	in-range
−1	2046	in-range
0	2047	in-range (= threshold)
+1	2048	in-range
+2	2049	in-range
±2047	4095, raw pixel	overflow

appears at least three times in a row) and **literal runs** (a count followed by that many values copied verbatim; the values need not be distinct). A single 16-bit header value tells the two apart using a midpoint constant $\text{midCount} = 32,767$ (half of the `uint16` range): a header $c \leq \text{midCount}$ means a same run of length c , and a header $c > \text{midCount}$ means a literal run of length $c - \text{midCount}$. The longest single run is therefore 32,767 symbols; longer stretches are split across several headers transparently.

As a concrete encoding example, the residual-code sequence $[7, 7, 7, 7, 7, 3, 9, 4]$ is emitted as the five 16-bit words $[5, 7, 32,770, 3, 9, 4]$: the header 5 ($\leq 32,767$) announces a same run of five, the word 7 is the repeated value, the header 32,770 ($= 32,767 + 3$) announces a literal run of three, and the three values follow verbatim.

Why 16-bit RLE matters. Suppose 1,000 identical 16-bit residuals appear in a row. MIC encodes that as just two numbers: a count and the value. A *byte-oriented* RLE sees the high and low bytes interleaved, so no single byte repeats 1,000 times, so it is forced into two interleaved runs or falls back to literals. Working at 16 bits captures the repetition that byte-splitting hides.

Fast decode. Same runs (often 80% of all runs on smooth images) are fast-pathed: the decoder just hands back the cached value and decrements a counter, without touching the compressed data at all.

D. The entropy coder (FSE) for large alphabets

FSE in one paragraph. FSE keeps a single integer “state.” To decode one symbol, it looks up the current state in a table; the entry tells it which symbol to output, how many bits to read next, and how to compute the next state. So each symbol costs **one table lookup, one bit read, and one addition**, with no multiplies or divides. (Encoding runs the symbols in reverse and builds the bitstream backwards; decoding reads forwards. That reversal is inherent to how ANS works.)

MIC extends FSE to handle up to **65,535 symbols** instead of Zstandard’s 4,096, because high-bit-depth residuals genuinely require a large alphabet; one code ($2^{16} - 1$) is reserved as the overflow delimiter of Section III-B. It also **sizes its tables to the data**: an 8-bit image stored in 16-bit containers (common for MR) shrinks the working tables from ~512 KB down to about 2 KB, which keeps everything in fast cache. This matters twice over for the browser targets, where the same table-sizing logic runs in JavaScript and WebAssembly memory.

Picking the table size automatically. FSE has a “tableLog” parameter that sets the size of the probability table: the table

has 2^{tableLog} entries, so each increment of `tableLog` doubles it. A larger table represents each symbol’s probability at finer granularity, which can improve the compression ratio, but it uses more memory and takes longer to build, and on a small or low-entropy image the extra precision is wasted. MIC sets `tableLog` from two measurements of the post-prediction RLE output stream that the entropy stage is about to encode:

- the **alphabet size** (written β , also called `symbolLen`): the number of distinct symbol values that actually occur in the observed RLE output; and
- the **symbol density** (written C_r): the total number of symbols in the RLE output fed to FSE (its `inputLength`) divided by the alphabet size β , i.e., the average amount of data backing each distinct symbol.

MIC uses the larger table (`tableLog` 12 instead of the default 11) only when *both* quantities are high enough, specifically more than 128 distinct symbols ($\beta > 128$) and more than 32 symbols of data per distinct value ($C_r > 32$). The intuition is that a finer table pays off only when the alphabet is genuinely large *and* each of its symbols is observed often enough to estimate the extra probabilities reliably; a large alphabet with little data per symbol (C_r small) would instead produce noisy frequency estimates and waste header space. On the test set this rule turned on the larger table for exactly the nine images that benefit (1–10% better ratio), with no effect on decode speed. Table IV shows the ablation across selected images.

TABLE IV
TABLELOG ABLATION: FORCED TL=11/12/13 VS. ADAPTIVE SELECTION (SELECTED IMAGES, ARM64 REFERENCE PLATFORM).

Image	TL=11	TL=12	TL=13	Adaptive
CR	3.47×	3.63×	3.69×	3.69×
MG1	8.00×	8.57×	8.79×	8.79×
MG2	7.98×	8.55×	8.77×	8.77×
MR2	3.14×	3.24×	3.28×	3.28×
RG2	3.85×	4.12×	4.23×	4.23×
RG3	5.64×	5.95×	6.08×	6.08×

IV. DECODE-SIDE PARALLELISM

Two mechanisms make the decoder fast enough for the viewer budget on heavy frames: multi-state FSE decoding, which extracts instruction-level parallelism inside one thread, and strip-parallel decoding (PICS), which extracts thread-level parallelism across cores or browser workers. Both are properties of the bitstream layout, so every decoder implementation in Section V benefits from them without format changes.

A. The single-chain bottleneck

A standard single-state ANS decoder has an inherent limitation: each symbol depends on the one before it. To decode symbol N the decoder needs the state produced by symbol $N - 1$, so the work cannot start early. With a table lookup taking roughly 4–5 CPU cycles, the loop is limited to about one symbol every few cycles, even though modern CPUs have 4–6 execution ports that remain largely idle. A single decode chain uses only one of them, leaving ~75–83% of the CPU’s capacity unused. That idle capacity is exactly what a viewer

needs to reclaim if client-side decode is to leave room for fetch and display inside a 25–60 FPS budget.

B. Multi-state interleaved decoding

MIC runs **several independent decoders at once** on interleaved positions. With 8 chains, chain 0 handles positions 0, 8, 16, ...; chain 1 handles 1, 9, 17, ...; and so on. The chains share nothing during decoding, each has its own state and its own stream of lookups, so the CPU’s out-of-order engine can keep all of them in flight simultaneously.

On the encoder side, the symbols are split into N interleaved groups, each compressed independently, and the resulting streams are concatenated with their boundary offsets recorded in the header. All chains share one FSE decode table, each chain’s starting state is recorded in the header, and the decoder writes each chain’s output back to its interleaved positions, so the reordering costs nothing beyond the writes the decoder performs anyway.

C. Measured speedup

In theory, N chains could be up to $N\times$ faster. In practice the speedup is smaller because the chains share some bookkeeping (refilling the bit reader), the unrolled loop puts pressure on the instruction cache, and reading the compressed data has its own bandwidth limit [25]. Measured on the FSE stage alone, **4 states** give +36% on ARM64 and +12% on AMD64 (relative to single state), and **8 states** a further +5% on ARM64 and +1% on AMD64, with **no change to the compressed file size for a given configuration**. ARM64 continues to improve as chains double; AMD64 flattens out earlier because its core already extracts substantial parallelism from a single chain. Section VIII-A shows that the optimum also depends on the runtime: JavaScript engines favor four states where native C favors eight. Table V shows the simple latency model against the measured ARM64 speedup.

TABLE V
MULTI-STATE DECODER LATENCY MODEL AND EMPIRICAL SPEEDUP.

States (k)	Amortized latency	Theoretical	Empirical (ARM64)
1	L cy/sym	1.0×	1.0×
2	$L/2$ cy/sym	2.0×	1.3×
4	$L/4$ cy/sym	4.0×	1.5×
8	$L/8$ cy/sym	8.0×	1.6×

D. Bitstream signalling and layout

The decoder signals its mode with a short magic header. In single-state mode the first byte encodes `tableLog`, whose maximum valid value is 17, so the byte value `0xFF` can never occur in a valid single-state stream; multi-state streams use `0xFF` as an extended-format marker followed by a tag byte for the chain count. Single-, two-, four-, and eight-state files therefore coexist without confusion, and one decoder can accept all of them; the JavaScript and WebAssembly decoders auto-detect the chain count per file. Table VI summarises the on-disk layout of a single-frame stream.

TABLE VI
SINGLE-FRAME MIC BITSTREAM LAYOUT. ALL MULTI-BYTE FIELDS ARE LITTLE-ENDIAN. $STATES$ k IS THE NUMBER OF INTERLEAVED FSE CHAINS ($k \in \{1, 2, 4, 8\}$).

Field	Size	Meaning
Magic / mode byte	1 B	Single-state: encodes <code>tableLog</code> (value 5–17). Multi-state: <code>0xFF</code> .
Multi-state tag	1 B	For multi-state streams, one of <code>0x02</code> , <code>0x04</code> , <code>0x84</code> indicating $k = 2, 4$, or 8 .
Symbol count	4 B	Number of decoded RLE symbols.
TableLog	1 B	Decode-table size: 2^{tableLog} entries (multi-state path only; the single-state path overloads byte 0).
Normalized counts	variable	FSE normalised symbol frequencies, serialised with the standard variable-length encoding.
Per-chain initial states	$k \times 2$ B	Starting state for each FSE chain.
Per-chain stream lengths	$k \times 4$ B	Compressed-stream length per chain (omitted when $k = 1$).
Per-chain compressed streams	variable	Concatenated chain payloads.

E. Strip-parallel decoding (PICS)

Parallel Image Compressed Strips (PICS) splits an image into N horizontal strips. Each strip is compressed independently with its own FSE table and its own predictor state, so each strip is independently decodable, and a strip offset table prepended to the payload lets a decoder dispatch strips to a worker pool: C POSIX threads (pthreads) natively, Web Workers or pthreads-on-WASM in the browser, and `worker_threads` under Node.js. Because the per-strip streams reuse the single-frame format of Table VI, the strip decoder is a thin scheduling layer over the same inner decoder used everywhere else.

Two properties of the format matter for the results later. First, per-strip FSE tables adapt to local statistics, so on large images the strip-split costs little ratio and occasionally improves it; on very small images the per-strip header overhead dominates and PICS is counterproductive (Section VII-E). Second, a strip whose data turns out to be incompressible is stored raw, flagged by the high bit of its length field; this guarantees the strip container never expands pathologically, at the cost of a visible size step when a high-entropy strip falls back (Section VIII notes one such case in the cine corpus).

V. IMPLEMENTATION VARIANTS AND DEPLOYMENT SCENARIOS

A. One bitstream, many decoders

The design principle behind the implementation family is strict format stability: every decoder variant consumes the identical compressed bytes and must reproduce the identical pixels. The evaluation harness enforces this by checksumming every decoded frame against a manifest generated from the source pixels (Section VI), so a performance difference between two variants can never be an artifact of decoding a different stream or of a silent precision loss. The practical consequence for a PACS deployment is that a study encoded once at the archive is served unchanged to every tier: the native gateway, the Node.js service, and the browser all decode the same object from storage, and switching a client from the pure JavaScript decoder to the WebAssembly decoder is a client-side rollout with no server-side re-encoding.

B. The variant matrix

Table VII lists the decoder variants measured in this paper. They differ along four axes: implementation language and runtime, the number of interleaved FSE states, the SIMD or threading acceleration applied, and the deployment tier they serve. None of the variants changes the compressed format family: multi-state streams are signalled in the header (Table VI) and PICS adds only the strip container of Section IV-E.

C. Deployment scenarios

The variants exist because real PACS deployments impose different constraints at different tiers. Table VIII summarises the six scenarios the evaluation covers, with the decoder artifact each one ships and its external dependencies.

D1: native gateway. A rendering gateway, archive node, or server-side viewer backend links the C decoder directly (or embeds the Go codec in a Go service). The C decoder is a single source file with no third-party dependencies, which keeps the integration surface small; the eight-state configuration with platform SIMD is the fastest single-threaded path, and PICS-C adds a bounded pthread pool for large single frames.

D2: zero-install web viewer. The pure JavaScript decoder is a ~20 KB minified ES module (about 24 KB including the parallel orchestrator and worker used by D5), with zero Node Package Manager (npm) dependencies. It requires no build step, no WASM MIME configuration, and no special response headers, so it deploys as a static file next to the viewer. It auto-detects 1/2/4/8-state streams and the PICS container. For institutions whose security review favors small auditable artifacts, a single-file decoder of about 1,700 source lines is a materially easier review target than a compiled multi-megabyte WASM library.

D3: optimized web viewer. The same single-file C decoder from D1 compiled to WebAssembly with Emscripten [11] yields a ~20 KB binary (plus ~10 KB of generated glue). It decodes the same bytes as D2 several times faster (Section VIII) and still requires nothing at run time beyond the browser’s WASM support. For comparison, the vendored WebAssembly builds of the baselines used by production viewers are about 2.0 MB (WASM) plus 123 KB (glue) for OpenJPH and about 142 KB plus 57 KB for CharLS.

D4: single-codebase WASM. The unmodified Go codec compiles to WebAssembly (`GOOS=js GOARCH=wasm`), producing a ~2.9 MB binary that bundles the Go runtime and garbage collector. This path exists to quantify the cost of the convenience: one codebase serving native and browser targets with no second implementation. The evaluation shows the cost is substantial (Section VIII), and this variant is reported as a measured negative result rather than a recommended deployment.

D5: threaded browser decode. Browsers permit shared-memory parallelism only in cross-origin isolated pages (the COOP/COEP response headers that enable `SharedArrayBuffer`). Under that condition MIC offers two strip-parallel paths over the same PICS files: a JavaScript Web Worker pool, and the C pthreads strip decoder compiled to WebAssembly with Emscripten’s pthreads support, where

TABLE VII

MIC DECODER VARIANTS: IMPLEMENTATION LANGUAGE, FSE DECODER CONFIGURATION, ACCELERATION, BITSTREAM COMPATIBILITY, AND DEPLOYMENT TIER. EVERY VARIANT REPRODUCES BIT-IDENTICAL PIXELS FROM THE SAME FORMAT FAMILY; “SAME BYTES” MEANS THE VARIANT DECODES THE IDENTICAL FILES USED BY THE OTHER VARIANTS OF ITS ROW GROUP.

Variant	Language / runtime	FSE states	Acceleration	Bitstream	Deployment tier
MIC-Go	Go (native)	2	none	reference format	native, any Go platform
MIC-4state (Go)	Go (native)	4	none	same container, k signalled	native, any Go platform
MIC-C	C via CGO	2	none	same bytes as MIC-Go	native server
MIC-SIMD	C via CGO	2	AVX-512 RLE/delta (AMD64)	same bytes	native server
MIC-4state-C	C via CGO	4	none	same bytes as MIC-4state	native server
MIC-4state-SIMD	C via CGO	4	BMI2 kernel (AMD64)	same bytes	native server
MIC (8-state C)	C via CGO	8	AVX-512 RLE/delta (AMD64); scalar/NEON-width (ARM64)	same container	native server (headline column)
PICS- N (Go)	Go (native)	4/8 per strip	N goroutines	PICS strip container	native, any Go platform
PICS-C- N	C via CGO	4/8 per strip	N pthreads + per-strip SIMD	PICS strip container	native server
MIC-JS	JavaScript (ES module)	1/2/4/8 (auto-detect)	none	same bytes	browser, Node.js
MIC-PICS (JS)	JavaScript + Web Workers	4/8 per strip	N workers, SharedArrayBuffer	PICS strip container	browser, Node.js (<i>worker_threads</i>)
MIC-WASM (Go)	Go compiled to WASM	4	none	same bytes	browser
MIC-C-WASM	C compiled to WASM (Emscripten)	4/8	scalar (wasm32)	same bytes	browser
MIC-C-WASM-PICS	C pthreads compiled to WASM	4/8 per strip	pthreads on Web Workers	PICS strip container	browser (cross-origin isolated)

TABLE VIII

DEPLOYMENT SCENARIOS AND THE DECODER ARTIFACT EACH ONE SHIPS. ARTIFACT SIZES ARE THE DEPLOYED BYTES MEASURED IN THE ACCOMPANYING REPOSITORY (WASM BINARY PLUS ITS JAVASCRIPT GLUE WHERE APPLICABLE; THE JAVASCRIPT DECODER SIZE IS THE MINIFIED MODULE SET). “NONE” UNDER RUNTIME DEPENDENCIES MEANS NO THIRD-PARTY LIBRARY OR PACKAGE IS REQUIRED AT RUN TIME.

Scenario	Environment	Decoder	Artifact size	Runtime dependencies	Parallelism
D1: Archive gateway / server rendering	native server (ARM64/AMD64)	MIC 8-state C; PICS-C- N	one C file (~1,400 source lines)	none	pthreads (PICS)
D2: Zero-install web viewer	any modern browser	MIC-JS	~24 KB minified (three files)	none (zero npm packages)	main thread
D3: Optimized web viewer	browser with WASM	MIC-C-WASM	~30 KB (20 KB WASM + 10 KB glue)	none at run time	main thread
D4: Single-codebase WASM	browser with WASM	MIC-WASM (Go)	~2.9 MB	Go WASM runtime (bundled)	main thread
D5: Threaded browser decode	browser, cross-origin isolated	MIC-PICS (JS) or MIC-C-WASM-PICS	~24 KB / ~47 KB	none	Web Workers / pthreads
D6: Node.js gateway or edge	Node.js	MIC-JS <i>worker_threads</i>	+ ~24 KB	none	<i>worker_threads</i>

each POSIX thread is backed by a nested Web Worker. The threaded WASM build is about 30 KB of WASM plus 17 KB of glue.

D6: Node.js gateway. The identical JavaScript module runs under Node.js for server-side rendering, prefetch services, or edge functions, using *worker_threads* as the PICS pool. No native addon compilation is involved, which keeps the service deployable on any Node.js platform image.

D. Portability and dependency accounting

Portability here is concrete: the Go codec compiles for every platform the Go toolchain targets with no external libraries; the C decoder is one file of portable C with optional, compile-time-gated SIMD paths (the WebAssembly build automatically uses the scalar fallbacks); the JavaScript decoder runs on any engine supporting ES2017 modules. The encoder-side toolchain (Go) and the browser artifacts are built once and served statically. By contrast, the baseline codecs require either native library integration (CMake-built C++ libraries for OpenJPH and CharLS, linked in-process for the native comparison) or multi-hundred-kilobyte to multi-megabyte WASM ports for the browser. The complete MIC core is about 1.1k source lines of Go, with the C decoder at about 1,400 lines and the JavaScript decoder at about 1,700 lines; JPEG-LS reference implementations are around 5k lines and HTJ2K around 20k (Table I), which is the difference between a reviewable codec and a trusted-because-standard one.

VI. EVALUATION METHODOLOGY

Dataset (systems benchmark, not clinical validation). 39 de-identified DICOM images across eleven modalities (CT, MR, computed radiography (CR), X-ray (XR), mammography, digital radiography, nuclear medicine, PET, fluoroscopy, secondary capture, and tomosynthesis), drawn from the public NEMA WG-04 sample sets [30], the NEMA 1997 CD, a public breast-tomosynthesis case [31], and additional grayscale slices from public GDCM and TCIA collections. Table IX lists the images. The corpus is sized and selected for **modality diversity and viewer stress cases** (tiny PET frames through multi-megabyte mammography and radiography), in the spirit of systems image suites such as Kodak’s 24-image set and CLIC validation sets of roughly 41–61 images, rather than as a clinical-validation cohort of the scale used in large interchange studies [27]. Per-image and geometric-mean results should be read as evidence on this set; population-level claims require a larger multi-site corpus (Section IX).

Multi-frame cine corpus. In addition to the 39 single-frame images, the browser evaluation uses seven multi-frame studies: cardiac cine MR (16 frames, 256×256, 8-bit), X-ray angiography (XA, 12 frames, 512×512, 8-bit), a nuclear-medicine gated heart study (13 frames, 64×64, 16-bit), an enhanced/volumetric MR study (10 frames, 64×64, 16-bit), an enhanced CT study (2 frames, 512×512, 16-bit), a breast tomosynthesis stack (16 frames, 1890×2457, 10-bit), and a CT axial series (16 frames, 512×512, 12-bit). The sources are public-domain sample DICOMs; each study is encoded as independent per-frame MIC bitstreams and played back client-

TABLE IX
TEST DATASET: 39 CLINICAL DICOM IMAGES SPANNING ELEVEN MODALITIES.

Image	Modality	Dimensions	Raw (MB)
MR	Brain MRI	256×256	0.13
CT	CT scan	512×512	0.50
CR	Comp. radiography	2140×1760	7.18
XR	X-ray	2048×2577	10.1
MG1	Mammography	2457×1996	9.35
MG2	Mammography	2457×1996	9.35
MG3	Mammography	4774×3064	27.3
MG4	Mammography	4096×3328	26.0
CT1	CT scan	512×512	0.50
CT2	CT scan	512×512	0.50
MG-N	Mammography	3064×4664	27.3
MR1	Brain MRI	512×512	0.50
MR2	Brain MRI	1024×1024	2.00
MR3	Brain MRI	512×512	0.50
MR4	Brain MRI	512×512	0.50
NM1	Nuclear med.	256×1024	0.50
RG1	Radiography	1841×1955	6.86
RG2	Radiography	1760×2140	7.18
RG3	Radiography	1760×1760	5.91
SC1	Sec. capture	2048×2487	9.71
XA1	Fluoroscopy	1024×1024	2.00
CT_BRAIN	CT scan	512×512	0.50
CT_ANKLE	CT scan	512×512	0.50
CT_ORT	CT scan	512×512	0.50
CT_CHEST	CT scan	400×512	0.39
MR_HEAD	Brain MRI	256×256	0.13
MR_INTERA	Brain MRI	1024×1024	2.00
DX_HAND	Digital radiography	1480×1410	3.98
DX_CHEST	Digital radiography	1416×1420	3.84
CR_THORAX	Comp. radiography	2076×1876	7.43
PET1	PET	256×256	0.13
PET2	PET	256×256	0.13
CT_LUNG	CT scan	512×512	0.50
CT_PANCREAS	CT scan	512×512	0.50
MR_BRAIN	Brain MRI	320×256	0.16
MR_BREAST	Breast MRI	256×256	0.13
MR_PROSTATE	Prostate MRI	320×320	0.20
PET_PSMa	PET	200×200	0.08
PET_LUNG	PET	200×200	0.08

side, matching the fetch–decode–display loop of Section I-A.

Baselines. HTJ2K (via OpenJPH [28]), JPEG-LS (via CharLS [29]), and Delta + Zstandard [18] at its maximum level. For the native tables all baselines were called in-process (no file or subprocess overhead) for a fair speed comparison. For the browser tables HTJ2K and JPEG-LS decode live through the same vendored WebAssembly builds used by production web viewers, reading their own .jph and .jls files, each of which was round-trip-verified with the native library before being admitted to the test set. JPEG XL appears as an informational row only (Section II).

Metrics. Compression ratio (uncompressed ÷ compressed), decode and encode throughput in MB/s of uncompressed pixels, browser decode latency per frame and cine frames per second, and modelled time-to-display.

Benchmark procedure. Native single-frame tables use two reference machines: an ARM64 host (Apple M4 Pro, 14 cores) and an AMD64 host (AWS EC2 c8i.4xlarge, Intel Xeon 6). Go code used the standard gc compiler with default optimization; the CGO-linked C components were built with -O3 (plus -march=native on AMD64). These are separate compilers with separate flag sets. Each native benchmark ran the full image ten times (-benchtime=10x, i.e., ten iterations, not ten seconds); run-to-run variance stayed under 2% on ARM64 and 5% on AMD64. JavaScript micro-benchmarks (Tables XVI and XVII) ran under Node.js v22.16 on an ARM64

laptop (Apple M2 Max), which also serves as the browser host: all in-browser measurements (Tables XVIII, XX, and XIX) ran under headless Chromium on that machine, with eight timed iterations and two warm-up iterations per codec and frame. The WebAssembly builds were produced with Emscripten at -O3. Elsewhere in the paper, platforms are referred to only as ARM64 or AMD64, and the browser host is called the ARM64 browser host.

Pixel-correctness verification. The browser harness decodes every frame with every live codec and checksums the decoded pixel buffer against a manifest generated from the source pixels at encode time. A run fails if any live decoder produces even one differing pixel. The harness is driven both interactively and by a headless continuous-integration runner; all browser numbers reported here come from runs in which every live decode verified bit-exact.

The “MIC” decoder column in native tables. The single “MIC” column in the native result tables always refers to the fastest available C decoder configuration on each platform, so that the comparison against the baselines is C-against-C rather than mixing a pure-Go decoder with optimized C libraries. Concretely, this is the eight-state decoder (Section IV). The entropy stage is identical across platforms; what differs is the vector width applied to the two stages that follow it. On AMD64 the run-length expansion and the prediction-reversal (delta-inverse) passes are accelerated with AVX-512; on ARM64 the same passes run with NEON-width or scalar code. This is purely a code-generation difference: both builds read the *identical* compressed bytes and produce bit-identical output. The other named variants of Table VII appear where the deployment comparison requires them.

Network model. Time-to-display for one image is modelled as

$$T = RTT + \frac{8 B_c}{r} + T_{dec}, \quad (1)$$

where B_c is the compressed size in bytes, r the link rate in bits per second, RTT the round-trip latency, and T_{dec} the measured client decode time. The eight network profiles in Table X span the deployments a PACS viewer encounters, from a wired radiology workstation to a satellite backhaul. The model deliberately excludes server think-time and rendering, which are codec-independent.

TABLE X
NETWORK PROFILES USED IN THE TIME-TO-DISPLAY MODEL.

Profile	Rate (Mbps)	RTT (ms)	Typical use
Gigabit LAN	1000	1	wired radiology workstation
Hospital LAN	100	2	wired clinical workstation
Hospital Wi-Fi	20	10	mobile cart / tablet on-site
Home broadband	25	20	remote read over VPN
5G	100	20	remote read, mobile hotspot
Cellular (4G/LTE)	5	60	on-call physician
3G	1.5	100	legacy cellular fallback
Congested / satellite	1	600	worst case backhaul

VII. RESULTS: RATIO AND NATIVE THROUGHPUT

A. Compression ratio

JPEG-LS produces the smallest files on nearly every image (37 of 39); its context-adaptive predictor is difficult to beat on

pure ratio. For a web PACS viewer, ratio is only one axis of the objective: smaller files reduce fetch time, but decode speed determines whether multi-frame scrolling stays interactive. MIC therefore targets the **ratio–speed–deployability balance** required by client-side viewing rather than archival size alone. Across the dataset MIC reaches a **3.36×** geometric-mean ratio: about 88% of JPEG-LS (3.84×) and within about 2% of HTJ2K (3.42×), while decoding faster than both. Mammography compresses best (up to 8.79×) because its smooth tissue produces long near-zero runs, and studies with large uniform backgrounds reach the highest single-image ratios (PET_PSMA at 10.12×, CT_ANKLE at 9.48×). The PICS-4 and PICS-8 columns show the strip-parallel format’s ratio cost: negligible on large images (sometimes slightly better, because per-strip FSE tables adapt locally) and prohibitive on the two 0.08 MB PET images, where per-strip overhead drives the ratio toward 1 and strips fall back to raw storage. Table XI gives the per-image ratios for all variants.

B. Encoding speed

MIC encodes faster than the others on most images, on both platforms, because its encoder is a single pass (predict, run-length, table-driven entropy code) while HTJ2K and JPEG-LS do more work per pixel. On **ARM64**, MIC is fastest on 38 of 39 images: roughly 1.0–3.3× HTJ2K, 1.0–7.1× JPEG-LS, and 30–320× Delta + Zstandard (whose max-level LZ77 search accounts for its low throughput); HTJ2K is ahead only on the tiny PET_LUNG study. On **AMD64**, MIC is fastest on 32 of 39; HTJ2K leads on the two largest mammography images, one radiography image, and four of the smaller studies, because the AMD64 baselines are themselves heavily vectorized and per-image table-build overhead weighs more on tiny images. Encoding throughput matters for the archive side of the viewer workflow (ingest and bulk migration); it is not on the per-frame display path. Table XII gives the per-image encoding throughput.

C. Decoding speed

Decoding is where MIC performs best. Its decoder is dominated by the compact FSE loop (one lookup, one bit read, one add) with no hard-to-predict branches. HTJ2K and JPEG-LS both have data-dependent branches that cause CPU mispredictions. Zstandard is the most comparable codec (same predictor, also uses FSE) but works on bytes, so it pays the 2× byte-splitting cost on wide data. On **ARM64**, MIC is fastest on all 39 images: roughly 1.2–2.1× HTJ2K, 1.0–2.0× Zstandard, and 1.1–5.1× JPEG-LS, with the largest margins on the smaller images. On **AMD64**, MIC is fastest on 36 of 39 images; on the three exceptions (CT_BRAIN, MR_INTERA, and the tiny PET_LUNG study) in-process Zstandard is marginally faster. Across its wins MIC runs about 1.0–1.8× HTJ2K and 1.9–7.1× JPEG-LS, and stays within 0.7–1.8× of Zstandard.

MIC therefore delivers both a **better ratio than Zstandard on most images** (34 of 39) and **faster decoding on nearly every image**, which reflects the 16-bit-native design. JPEG-LS retains about a 14% ratio advantage, so the archival choice between them is decode speed versus storage cost; for

interactive viewing the decisive quantity is whether per-frame decode time leaves room for fetch and display at 25–60 FPS (Section VIII). Per-image numbers are in Table XIII, and the entropy stage in isolation is in Table XIV.

D. The entropy stage in isolation

Table XIV isolates the FSE stage from the delta and RLE stages: the input is the RLE symbol stream and MB/s is measured over that uncompressed symbol stream. The 1-, 4-, and 8-state variants are all pure Go; the only difference is how many independent FSE state chains the decoder interleaves per inner-loop iteration. This is a parallel microbenchmark (iterations decode concurrently across cores), so the absolute MB/s values are aggregate figures and are not comparable with the single-thread end-to-end numbers of Table XIII; the quantity of interest is the *relative* gain from adding state chains, which isolates the instruction-level-parallelism effect that Section IV predicts.

E. Multi-threaded decoding (PICS)

For large single frames that would otherwise miss the viewer frame-rate target, the PICS strip decoder (Section IV-E) dispatches strips to a C pthreads pool. This scales nearly linearly up to four threads on both platforms and keeps scaling to eight threads on images above ~1 MB, reaching about **4.3 GB/s** on the largest mammography images on ARM64. Small images do not benefit: thread setup costs more than it saves, and on the two 0.08 MB PET images the strips fall back toward raw storage, so strip parallelism does not apply. We report this separately because the other codecs run single-threaded in our pipeline, so a direct comparison would not be fair. Table XV gives the per-image strip-parallel numbers.

Practical guidance: to decode many small images, use single-threaded MIC per request, which is already fast. To decode one large image on a multi-core machine, use strip-parallel mode with four to eight threads.

VIII. RESULTS: CROSS-DEPLOYMENT EVALUATION

This section evaluates the deployment scenarios of Section V-C on their own terms: the same compressed objects, decoded by every implementation tier, with every live decode verified bit-exact against the source pixels.

A. Node.js (scenario D6)

The pure JavaScript decoder was first characterized under Node.js, which shares the V8 engine with Chromium-based browsers. Notably, the **four-state** variant runs 6–21% *faster* than eight-state in JavaScript, even though eight states win in C on the same hardware. The reason is the JavaScript engine: it can keep four independent chains busy but not eight, so the extra chains add overhead without shortening the critical path. The eight-state path still works in JavaScript (one decoder accepts C-encoded files without conversion), but four states is the best configuration for JavaScript engines. This is a concrete example of why the deployment matrix has to be measured

TABLE XI
LOSSLESS COMPRESSION RATIOS: ALL CODEC VARIANTS, 39 IMAGES. BOLD = BEST RATIO PER ROW.

Image	Raw (MB)	Δ +Zstd-19	MIC	Wavelet	PICS-4	PICS-8	HTJ2K	JPEG-LS
MR	0.13	1.95×	2.35×	2.38×	2.28×	2.21×	2.38×	2.52×
CT	0.50	2.05×	2.24×	1.67×	2.15×	1.96×	1.77×	2.68×
CR	7.18	3.24×	3.69×	3.81×	3.70×	3.71×	3.77×	3.96×
XR	10.1	1.43×	1.74×	1.76×	1.75×	1.75×	1.67×	1.76×
MG1	9.35	7.20×	8.79×	8.66×	8.84×	8.87×	8.25×	8.91×
MG2	9.35	7.21×	8.77×	8.65×	8.83×	8.85×	8.24×	8.90×
MG3	27.3	2.09×	2.24×	2.31×	2.31×	2.33×	2.22×	2.38×
MG4	26.0	3.10×	3.47×	3.59×	3.58×	3.62×	3.51×	3.71×
CT1	0.50	2.47×	2.79×	2.48×	2.54×	2.29×	2.70×	3.19×
CT2	0.50	3.24×	3.48×	2.87×	3.11×	2.72×	3.29×	4.54×
MG-N	27.3	2.04×	2.24×	2.32×	2.31×	2.34×	2.23×	2.38×
MR1	0.50	1.79×	2.09×	2.14×	2.10×	2.08×	2.12×	2.30×
MR2	2.00	2.77×	3.28×	3.34×	3.31×	3.31×	3.35×	3.52×
MR3	0.50	3.47×	3.92×	4.09×	3.89×	3.83×	4.33×	4.51×
MR4	0.50	3.58×	4.12×	4.18×	4.09×	4.03×	4.21×	4.49×
NM1	0.50	4.88×	5.15×	5.01×	5.25×	5.28×	5.76×	6.28×
RG1	6.86	1.45×	1.70×	1.70×	1.70×	1.69×	1.63×	1.72×
RG2	7.18	3.69×	4.23×	4.32×	4.28×	4.30×	4.32×	4.51×
RG3	5.91	5.46×	6.08×	6.82×	6.11×	6.12×	6.99×	7.31×
SC1	9.71	3.46×	3.71×	3.70×	3.73×	3.73×	3.85×	4.73×
XA1	2.00	4.37×	5.01×	4.94×	5.04×	5.03×	4.88×	5.39×
CT_BRAIN	0.50	3.08×	3.06×	2.89×	2.77×	2.47×	3.39×	4.28×
CT_ANKLE	0.50	9.30×	9.48×	5.02×	9.30×	9.14×	4.97×	6.87×
CT_ORT	0.50	2.36×	2.68×	2.38×	2.51×	2.26×	2.56×	3.00×
CT_CHEST	0.39	2.46×	2.82×	2.10×	2.52×	2.21×	2.23×	3.22×
MR_HEAD	0.13	2.11×	2.61×	2.65×	2.57×	2.53×	2.67×	2.86×
MR_INTERA	2.00	4.71×	3.68×	4.05×	3.75×	3.78×	5.04×	5.08×
DX_HAND	3.98	1.99×	2.24×	2.35×	2.26×	2.25×	2.37×	2.48×
DX_CHEST	3.84	1.43×	1.66×	1.82×	1.65×	1.63×	1.63×	1.70×
CR_THORAX	7.43	1.61×	1.82×	1.81×	1.84×	1.83×	1.81×	1.90×
PET1	0.13	2.37×	2.74×	2.81×	2.64×	2.52×	3.02×	3.21×
PET2	0.13	3.39×	3.39×	3.48×	3.16×	2.58×	4.37×	4.74×
CT_LUNG	0.50	2.40×	2.73×	2.45×	2.50×	2.25×	2.67×	3.15×
CT_PANCREAS	0.50	2.09×	2.36×	1.76×	2.18×	1.98×	1.85×	2.70×
MR_BRAIN	0.16	6.17×	7.27×	6.75×	7.31×	7.24×	7.62×	8.46×
MR_BREAST	0.13	3.31×	4.01×	3.94×	4.00×	3.91×	4.10×	4.48×
MR_PROSTATE	0.20	2.04×	2.30×	2.34×	2.28×	2.26×	2.49×	2.65×
PET_PSMA	0.08	11.88×	10.12×	7.67×	1.20×	1.18×	13.91×	15.78×
PET_LUNG	0.08	4.56×	2.97×	2.97×	1.00×	1.03×	5.21×	5.62×
Geomean	—	3.06×	3.36×	3.21×	3.04×	2.95×	3.42×	3.84×

rather than extrapolated from native numbers: the optimal codec configuration changes with the runtime.

Concretely, a 9.35 MB mammography image decodes in 17.1 ms (547 MB/s) using four-state strips across `worker_threads`. Tables XVI and XVII report the Node.js micro-benchmarks on the ARM64 browser host (Section VI).

Practitioner takeaway. For simplicity in a pure browser or Node.js path, ship the four-state JavaScript decoder (and optional PICS workers). For maximum single-threaded server or gateway throughput, use MIC-4state-C or the eight-state C decoder as appropriate on the host. For large frames on multicore clients (tomosynthesis, full-field mammography), use PICS with four to eight strips so the decode stage stays inside the 25–60 FPS budget.

B. In-browser single-frame comparison (scenarios D2–D5)

Table XVIII shows the deployment matrix distilled to one representative heavy image: the 7.18 MB CR radiograph, decoded live in headless Chromium on the ARM64 browser host. All MIC rows decode *identical bytes*; the reference codecs decode their own files. Three findings stand out.

First, **implementation tier dominates decode time at equal bytes**: across the five MIC rows the identical compressed input decodes anywhere from ~330 ms to ~4 ms, an 80× spread that

is entirely decoder-implementation cost. The C-to-WASM build is about 8× faster than the pure JavaScript decoder from a 20 KB binary, and the threaded C-to-WASM build multiplies that by the worker count, reaching ~1.8 GB/s inside a browser tab.

Second, **compiling a managed-runtime codec to WebAssembly can be slower than tuned JavaScript**: the Go-to-WASM build, which bundles the Go runtime and garbage collector, decodes at roughly 0.4× the speed of the hand-written JavaScript decoder despite being 120× larger. The single-codebase convenience of scenario D4 is real, but its performance and payload cost make it the wrong default for a viewer.

Third, the baselines are competitive in the mid-tier: **the CharLS WebAssembly decoder is about twice as fast as MIC’s pure JavaScript decoder on this image**, and OpenJPH’s is comparable to it. The like-for-like comparison, WASM against WASM, favors MIC by about 4× over CharLS and 7× over OpenJPH single-threaded, and by far more once strips and workers are engaged, which no browser build of the baselines offered at the time of writing.

C. Multi-frame cine playback across the full matrix

Table XIX reports the full deployment matrix on the seven multi-frame cine studies: twelve decoder configurations, each

TABLE XII
SINGLE-THREADED ENCODING THROUGHPUT (MB/s). MIC IS THE EIGHT-STATE C ENCODER. BOLD MARKS THE FASTEST CODEC PER ROW ON EACH PLATFORM.

Image	ARM64				AMD64			
	MIC	Δ +Zstd	HTJ2K	JPEG-LS	MIC	Δ +Zstd	HTJ2K	JPEG-LS
MR	457	7	236	91	344	5	246	62
CT	476	7	183	126	280	5	207	97
CR	525	2	186	110	322	2	286	75
XR	661	3	229	113	400	4	256	84
MG1	1,057	11	414	277	532	8	632	211
MG2	1,047	12	407	284	552	8	629	209
MG3	639	2	211	132	352	2	245	104
MG4	883	5	342	196	457	6	408	151
CT1	615	9	243	176	321	7	271	125
CT2	536	7	204	172	285	5	271	124
MG-N	639	2	214	140	358	2	248	104
MR1	616	9	205	114	367	6	241	86
MR2	685	7	216	118	409	4	309	85
MR3	768	19	264	167	441	12	354	123
MR4	585	7	212	163	367	5	312	142
NM1	633	6	208	162	340	5	313	115
RG1	595	4	218	84	396	5	252	66
RG2	737	4	263	151	425	3	360	100
RG3	757	6	276	183	389	4	412	126
SC1	667	5	225	203	416	4	306	169
XA1	612	5	203	133	356	4	303	98
CT_BRAIN	352	5	168	147	239	4	253	112
CT_ANKLE	955	15	290	327	450	11	358	261
CT_ORT	644	10	198	138	350	7	269	112
CT_CHEST	463	6	177	133	267	4	234	100
MR_HEAD	430	6	211	106	288	4	252	65
MR_INTERA	415	4	180	115	269	3	268	89
DX_HAND	587	4	218	102	378	3	261	65
DX_CHEST	636	6	209	109	389	6	241	77
CR_THORAX	585	4	226	91	397	4	261	68
PET1	479	7	240	129	295	5	252	111
PET2	550	6	236	145	269	4	256	103
CT_LUNG	586	10	195	141	337	7	258	110
CT_PANCREAS	454	7	193	130	292	5	240	98
MR_BRAIN	607	7	229	191	338	6	351	141
MR_BREAST	522	5	241	141	292	4	290	96
MR_PROSTATE	465	8	195	97	313	6	250	66
PET_PSMA	434	14	423	426	296	10	377	364
PET_LUNG	232	6	245	235	144	5	260	167
Geomean	582	6	231	148	343	5	293	110

decoding every frame of every study in headless Chromium, reported as sustained cine frames per second. Table XX gives the corresponding compressed payloads for the codec-level comparison.

Four observations follow. First, **every MIC browser configuration except Go-WASM meets the 25–60 FPS target on six of the seven studies**; the exception in the matrix is breast tomosynthesis (DBT), whose ~ 9.3 MB frames are comparable to a full mammography image. There, pure JavaScript sustains ~ 18 FPS, below the target band; the JavaScript worker pool recovers 90 FPS, single-threaded C-WASM reaches 66–67 FPS, and the threaded C-WASM build reaches 252 FPS, an order of magnitude of headroom over the target. The heaviest clinically realistic case in the corpus is thus decidable in the browser well above cine rates, but only by the variants designed for it.

Second, **parallelism has a floor**: on the two 64×64 studies (~ 8 KB raw per frame) the worker pools are *slower* than single-threaded decoding because dispatch and collection overhead exceeds the decode work itself. Single-threaded MIC-C-WASM is the right configuration there (8,400–8,500 FPS on the gated heart study). Strip parallelism pays off from roughly the 512×512 class upward.

Third, among the baselines, **JPEG-LS via CharLS-WASM is a strong mid-tier browser decoder** (it exceeds the

pure-JavaScript MIC rows on five studies and produces the smallest live-decodable files throughout), while HTJ2K via OpenJPH-WASM trails on every study and drops to 9 FPS on tomosynthesis. If a deployment restricts itself to standardized DICOM transfer syntaxes with existing WASM decoders, JPEG-LS rather than HTJ2K is the browser-side performance choice on this evidence; MIC’s C-WASM tiers are 4–7 $\times$ faster still.

Fourth, **JPEG XL would be the ratio winner if it were feasible**: its files are the smallest on all seven studies (Table XX), but with no lossless 16-bit browser decode path at the time of writing, that ratio is unusable for this workflow without server-side transcoding, which is exactly what the client-side architecture is designed to avoid.

D. Time-to-display across network profiles

Decode time is only part of the user-visible latency. Applying the model of Eq. (1) to the measured browser decode times and compressed sizes of the CR radiograph yields Table XXI: modelled time-to-display for five decoder configurations across five representative network profiles (the remaining three profiles of Table X interpolate between the columns shown).

The pattern is the paper’s second headline result. **On fast networks, decoder choice dominates**: on a Gigabit LAN

TABLE XIII

SINGLE-THREADED DECOMPRESSION THROUGHPUT (MB/s). MIC IS THE EIGHT-STATE C DECODER (AVX-512 ACCELERATION OF THE RLE AND DELTA INVERSE STAGES ON AMD64). BOLD MARKS THE FASTEST CODEC PER ROW ON EACH PLATFORM.

Image	ARM64				AMD64			
	MIC	Δ +Zstd	HTJ2K	JPEG-LS	MIC	Δ +Zstd	HTJ2K	JPEG-LS
MR	649	389	328	137	614	360	453	91
CT	495	406	299	158	444	393	329	110
CR	580	527	316	181	570	479	445	124
XR	668	512	324	131	626	509	393	88
MG1	762	685	612	481	916	596	913	336
MG2	753	692	622	492	942	599	902	334
MG3	600	433	314	175	547	362	392	118
MG4	788	504	508	246	686	469	634	158
CT1	600	450	357	224	535	432	389	142
CT2	570	471	342	212	522	458	383	131
MG-N	600	408	289	180	549	357	398	118
MR1	734	365	342	149	586	381	392	93
MR2	700	445	363	211	682	460	486	131
MR3	839	523	432	289	742	497	520	192
MR4	692	466	326	220	664	496	433	152
NM1	778	499	384	260	597	515	485	171
RG1	469	383	318	123	547	340	363	80
RG2	652	510	383	225	716	503	544	154
RG3	685	642	432	294	719	564	641	195
SC1	699	511	366	263	673	483	457	184
XA1	662	501	346	244	670	541	501	165
CT_BRAIN	528	466	326	190	420	477	354	122
CT_ANKLE	853	587	452	426	774	562	537	282
CT_ORT	538	442	348	190	565	430	384	124
CT_CHEST	500	445	297	178	446	428	333	110
MR_HEAD	595	372	280	167	679	371	406	100
MR_INTERA	590	563	304	178	454	527	436	133
DX_HAND	500	414	316	147	570	416	362	101
DX_CHEST	476	352	307	121	547	349	358	83
CR_THORAX	552	403	346	129	561	365	365	84
PET1	732	416	377	197	679	384	389	120
PET2	656	515	381	227	487	438	389	141
CT_LUNG	522	316	348	189	534	420	383	122
CT_PANCREAS	517	451	316	172	439	390	337	110
MR_BRAIN	866	567	422	375	708	514	641	221
MR_BREAST	640	545	350	221	759	449	510	156
MR_PROSTATE	679	422	353	165	545	386	408	103
PET_PSMa	759	678	475	684	747	655	539	395
PET_LUNG	502	498	283	278	365	508	363	170
Geomean	631	473	359	215	598	452	445	140

TABLE XIV

FSE DECOMPRESSION THROUGHPUT (MB/s) OF THE ENTROPY STAGE IN ISOLATION, PURE-Go DECODER, AGGREGATE PARALLEL MICROBENCHMARK: THE RELATIVE GAIN ACROSS 1/4/8 INTERLEAVED STATE CHAINS IS THE QUANTITY OF INTEREST. PER-IMAGE ROWS ARE TEN REPRESENTATIVE MODALITIES; BOTH GEOMETRIC MEANS ARE OVER ALL 39 IMAGES.

Image	ARM64 (MB/s)			AMD64 (MB/s)		
	1-st	4-st	8-st	1-st	4-st	8-st
MR	1,066	1,576	1,068	656	1,007	847
CT	1,001	1,549	2,142	972	1,034	1,362
CR	2,941	4,882	6,245	1,456	1,844	1,709
XR	3,597	6,081	6,030	1,745	2,341	2,014
MG1	3,558	4,865	4,885	2,216	1,487	1,975
MG-N	3,949	6,647	6,037	1,940	2,545	2,415
NM1	1,971	2,661	2,570	917	1,346	1,356
RG1	1,768	4,356	5,332	1,239	1,709	1,869
DX_HAND	2,120	3,163	4,816	1,245	1,833	1,520
PET1	1,105	1,252	1,530	575	763	616
Geomean (39)	1,742	2,372	2,492	1,093	1,228	1,246

the spread between the best and worst configuration is 21 ms versus 157 ms (7.5 $\times$), the difference between imperceptible and noticeable on every scroll step, and MIC's threaded WASM path is fastest. **On slow networks, compression ratio dominates:** on the 4G and satellite profiles all configurations land within

about 9% of each other, transfer swamps decode, and the smallest live-decodable file wins, which on this image is JPEG-LS. The crossover for this image sits around the hospital Wi-Fi / home broadband class, where transfer and decode are the same order of magnitude and both axes matter. A deployment can exploit this directly: the archive stores one MIC object and the viewer picks the decoder tier per client, while sites dominated by very slow links may prefer the highest-ratio codec their clients can decode, accepting the decode-speed penalty that is invisible behind the transfer time. On studies where JPEG XL's ratio advantage is larger (enhanced CT, cardiac cine MR; Table XX) a feasible browser decoder for it would win the slow-network regime outright, which quantifies what the missing 16-bit browser path costs in practice.

IX. DISCUSSION

A. Choosing a configuration per deployment scenario

The results reduce to concrete guidance per tier of Table VIII:

- **D1 (native gateway):** the eight-state C decoder is the fastest single-threaded configuration on both architectures (geometric mean 631 MB/s on ARM64, 598 MB/s on AMD64), and PICS-C with four to eight threads is the

TABLE XV

MULTI-THREADED DECOMPRESSION THROUGHPUT (MB/s) FOR MIC’S STRIP-PARALLEL MODE. THE “MIC” COLUMN REPRODUCES THE SINGLE-THREADED EIGHT-STATE BASELINE FROM TABLE XIII EXACTLY; N -STRIP COLUMNS DECODE THE SAME COMPRESSED BYTES IN N PARALLEL THREADS. BOTH ARM64 AND AMD64 COLUMNS COVER ALL 39 IMAGES.

Image	ARM64				AMD64			
	MIC	2-strip	4-strip	8-strip	MIC	2-strip	4-strip	8-strip
MR	649	698	939	1,033	614	281	369	291 [†]
CT	495	511	987	1,549	444	379	641	736
CR	580	865	1,771	3,227	570	768	1,404	2,236
XR	668	888	1,728	3,197	626	787	1,408	2,252
MG1	762	1,415	2,403	4,342	916	1,330	1,924	3,575
MG2	753	1,369	2,226	4,109	942	1,302	1,698	3,011
MG3	600	934	1,763	3,158	547	868	1,597	2,781
MG4	788	1,263	2,185	4,120	686	1,258	1,957	3,107
CT1	600	779	1,178	1,639	535	477	743	797
CT2	570	773	1,249	1,296	522	475	718	806
MG-N	600	988	1,852	3,569	549	871	1,644	2,913
MR1	734	931	1,375	1,898	586	517	729	827
MR2	700	1,003	1,699	2,750	682	786	1,064	1,476
MR3	839	919	1,381	2,035	742	627	861	853
MR4	692	954	1,301	2,063	664	573	813	877
NM1	778	964	1,410	1,868	597	561	777	771
RG1	469	586	1,068	1,964	547	626	1,113	1,833
RG2	652	1,095	1,887	3,269	716	922	1,523	2,494
RG3	685	1,081	2,027	3,289	719	991	1,596	2,504
SC1	699	1,171	2,005	3,174	673	1,042	1,690	2,598
XA1	662	973	1,692	2,404	670	806	1,170	1,443
CT_BRAIN	528	627	1,025	1,291	420	426	630	681
CT_ANKLE	853	1,082	1,745	1,713	774	654	869	877
CT_ORT	538	679	1,099	1,316	565	485	726	778
CT_CHEST	500	622	1,042	1,354	446	409	633	691
MR_HEAD	595	620	915	880	679	306	417	307
MR_INTERA	590	910	1,575	2,405	454	618	940	1,329
DX_HAND	500	605	1,113	2,051	570	619	1,133	1,697
DX_CHEST	476	566	1,003	1,971	547	592	990	1,712
CR_THORAX	552	588	1,128	1,733	561	624	1,141	1,937
PET1	732	637	854	854	679	289	406	317
PET2	656	669	776	923	487	341	356	320
CT_LUNG	522	689	1,107	1,462	534	468	752	808
CT_PANCREAS	517	506	1,018	1,316	439	390	652	688
MR_BRAIN	866	815	999	1,143	708	435	490	448
MR_BREAST	640	653	920	956	759	288	396	337
MR_PROSTATE	679	742	1,093	1,279	545	321	507	455
PET_PSMA [‡]	759	727	–	–	747	282	–	–
PET_LUNG [‡]	502	359	–	–	365	208	–	–

[†]The 130 KB MR image is too small to benefit from 8-way parallelism on AMD64: synchronization overhead exceeds the per-strip work, so 4-strip is faster than 8-strip. [‡]On the two 0.08 MB PET images, splitting into four or more strips drives the strip-compressed size to or above the single-stream size, so their 4- and 8-strip figures reflect near-raw copying rather than entropy decoding and are omitted.

TABLE XVI

JAVASCRIPT DECODER THROUGHPUT UNDER NODE.JS ON THE ARM64 BROWSER HOST (SECTION VI), FOR THE FOUR-STATE AND EIGHT-STATE FSE VARIANTS. MEDIAN OVER 20 ITERATIONS AFTER THREE WARM-UP RUNS.

Image	Dimensions	Ratio	4-state		8-state	
			ms	MB/s	ms	MB/s
MR [†]	256×256	2.35×	2.9	43	2.7	46
CT	512×512	2.24×	12.3	41	13.1	38
CR	2140×1760	3.69×	137.2	52	165.5	43
MG1	2457×1996	8.79×	70.4	133	80.0	117
MG3	4774×3064	2.29×	560.8	50	607.1	46
DX_HAND	1480×1410	2.24×	82.6	48	96.1	41
PET1	256×256	2.74×	2.1	60	2.5	50

[†]MR’s 0.13 MB image decodes in under 3 ms, at the JavaScript timing floor; the four-state advantage seen on the larger images is within run-to-run noise here.

choice for large single frames (up to about 4.3 GB/s on mammography-sized images on ARM64). For many small images in parallel requests, per-request single-threaded decoding already saturates the machine without strip overhead.

TABLE XVII

PICS PARALLEL JAVASCRIPT DECODER (`WORKER_THREADS`) UNDER NODE.JS ON THE ARM64 BROWSER HOST (SECTION VI), WITH FOUR-STATE AND EIGHT-STATE FSE PER STRIP. WORKER COUNT WAS SWEEPED OVER {1, 2, 4, 8, 14}; THE BEST WALL-TIME FOR EACH VARIANT IS REPORTED.

Image	Strips	4-state strips		8-state strips	
		ms	MB/s	ms	MB/s
MR	4	0.9	134	0.9	135
CT	4	4.1	123	4.5	112
CR	8	22.9	313	28.0	257
MG1	8	17.1	547	16.7	559
DX_HAND	8	17.1	233	17.8	224

- **D2 (zero-install browser):** the four-state pure JavaScript decoder is the correct default; eight-state files still decode but are 6–21% slower under JavaScript engines. It meets the cine target on everything in the corpus except tomosynthesis-sized frames.
- **D3 (WASM browser):** the C-to-WASM decoder is the best performance-per-byte in the study: about 8× the JavaScript decoder from a ~30 KB artifact, and the fastest single-

TABLE XVIII

LIVE IN-BROWSER DECODE OF ONE 7.18 MB CR RADIOGRAPH (1760×2140) ON THE ARM64 BROWSER HOST (HEADLESS CHROMIUM): DECODER IMPLEMENTATION COST AT EQUAL BYTES. ALL MIC ROWS DECODE THE SAME FOUR-STATE FILE (PICS ROWS DECODE ITS STRIP-CONTAINER EQUIVALENT); EVERY ROW IS PIXEL-VERIFIED BIT-EXACT. ARTIFACT SIZE IS THE DEPLOYED DECODER (WASM + GLUE, OR MINIFIED JAVASCRIPT).

Decoder	Artifact	Decode (ms)	MB/s
MIC-JS (4-state, D2)	~24 KB	~140	~51
MIC-WASM, Go (D4)	~2.9 MB	~330	~22
MIC-C-WASM (4-state, D3)	~30 KB	~17	~422
MIC-PICS (JS, 8 workers, D5)	~24 KB	~24	~299
MIC-C-WASM-PICS (8 strips, D5)	~47 KB	~4	~1,795
HTJ2K (OpenJPH WASM)	~2.1 MB	~125	~57
JPEG-LS (CharLS WASM)	~199 KB	~70	~103

threaded browser path measured, including the baselines.

- **D4 (Go-to-WASM):** measured as a negative result. Roughly 0.4× the pure JavaScript decoder at 120× the payload. Useful only where reusing the Go codebase unmodified outweighs performance.
- **D5 (threaded browser):** for tomosynthesis and full-field mammography, the C-pthreads WASM strip decoder is the only configuration with a large margin above the target band (252 FPS on DBT frames); the JavaScript worker pool is the fallback when shipping WASM is not an option. Neither helps below roughly 512×512 frames, where worker dispatch overhead exceeds decode work.
- **D6 (Node.js):** the same JavaScript module with `worker_threads` reaches 230–560 MB/s on large images, enough for server-side prefetch or thumbnail services without any native addon.

B. The remaining variants

The main tables use one MIC column per platform, but the codebase includes variants that exist for deployment reasons rather than to compete on raw speed. The **pure Go** build runs the entire pipeline with no C dependency at roughly 40–60% of the C decoder’s speed; it is the natural choice inside Go services and on platforms where CGO is unavailable, and it is the same code that compiles to the D4 WebAssembly build. The **1/2/4-state decoders** are the same decoder with fewer chains and serve as baselines for the multi-state comparison; four states is the JavaScript optimum, eight the native optimum. **Wavelet-based MIC** replaces the spatial predictor with a reversible 5/3 wavelet transform [23] feeding the same RLE and FSE back end; its ratio column in Table XI shows it competitive on a few noisy radiographs (XR, DX_CHEST) but weaker in geometric mean (3.21× versus 3.36×) and notably poor on full-range CT, where coefficient expansion forces frequent overflow escapes. On this corpus the added transform complexity does not pay for the viewer workload, which is why the deployed pipeline keeps the simpler predictor.

C. A codec selection matrix

A structured comparison, combining a Pareto view of ratio versus decode speed [32], [33] with a weighted decision matrix over ratio, speed, platform coverage, and deployment

ease [34], [35] following deployment taxonomies for DICOM compression [26], [36], puts MIC, HTJ2K, and JPEG-LS on the efficiency frontier (MIC for throughput, JPEG-LS for ratio, HTJ2K in between), with Delta + Zstandard dominated by MIC on both axes. HTJ2K remains the choice whenever standard compatibility is a hard requirement that the scoring does not capture, though the browser evidence of Section VIII suggests JPEG-LS over HTJ2K when the client is a browser. Table XXII reports the weighted matrix.

D. Limitations

Corpus size and statistical weight. The evaluation rests on 39 single-frame images and seven multi-frame studies. That is appropriate as a systems throughput suite (diverse enough to exercise smooth mammography, noisy CT, tiny PET, and multi-megabyte radiography), but it is far smaller than clinical-validation studies that may use thousands of images [27], and it is not a controlled multi-site archive sample. Several modalities appear in only two or three images, so per-modality means carry little statistical weight, and geometric means can move with a single outlier.

Single browser host. All in-browser numbers come from one ARM64 laptop under headless Chromium. The relative ordering of the decoder tiers (JavaScript versus C-WASM versus threaded WASM) rests on engine characteristics shared across Chromium platforms, but the absolute FPS values, and the JavaScript four-state versus eight-state optimum, should be re-measured on lower-powered clinical endpoints (ward workstations, tablets) and other engines before being treated as general.

Modelled network, measured decode. The time-to-display analysis combines measured decode times with an analytic transfer model (Eq. (1)). Real deployments add TLS handshakes, HTTP overheads, caching, and concurrent-transfer effects the model omits; the qualitative crossover between decode-bound and transfer-bound regimes is robust to these, the exact crossover point is not.

Held-out parameter tuning. The adaptive table-size thresholds ($\beta > 128$, $C_r > 32$) and the choice of the average predictor were informed by this same dataset, so there is a risk that they are fitted to it rather than to medical images in general. The rules are deliberately coarse (two integer thresholds and a single fixed predictor, with no per-image search), and they gate a modest 1–10% ratio gain on a minority of images, but a properly held-out study, with the thresholds frozen on one collection and measured on a disjoint one, is required to show they generalize.

E. Toward a DICOM transfer syntax

The evaluation operates on extracted pixel arrays and on browser-side `.mic` objects suitable for object-storage delivery to a web viewer. To route the same bitstream through a full PACS archive without a private side channel, MIC would need to be carried as a DICOM transfer syntax. The mapping is direct: MIC fits the existing encapsulated pixel-data model that JPEG 2000 and JPEG-LS already use (fragments with a Basic Offset Table in element 7FE0,0010), each frame’s self-contained bitstream (Table VI) stored as one or more

TABLE XIX

IN-BROWSER CINE DECODE RATE (FRAMES PER SECOND) FOR TWELVE DECODER CONFIGURATIONS ON SEVEN MULTI-FRAME STUDIES (ARM64 BROWSER HOST, HEADLESS CHROMIUM; EIGHT TIMED ITERATIONS AND TWO WARM-UP ITERATIONS PER CODEC AND FRAME; EVERY LIVE DECODE PIXEL-VERIFIED). MIC ROWS DECODE IDENTICAL BYTES PER STUDY (PICS ROWS DECODE THE STRIP-CONTAINER VARIANT). THE 25–60 FPS BAND IS THE INTERACTIVE VIEWING TARGET.

Decoder	Cardiac MR 16f, 256 ²	XA angio 12f, 512 ²	NM heart 13f, 64 ²	Enh. MR 10f, 64 ²	Enh. CT 2f, 512 ²	DBT 16f, 1890×2457	CT series 16f, 512 ²
MIC-JS, 1-state	300	136	1,045	672	174	19	102
MIC-JS, 4-state	300	140	937	607	197	18	101
MIC-JS, 8-state	286	140	925	847	181	18	97
MIC-WASM (Go)	153	46	502	513	46	3	44
MIC-C-WASM, 4-state	1,077	430	8,497	6,849	658	66	554
MIC-C-WASM, 8-state	954	396	8,442	7,547	548	67	483
MIC-PICS (JS, 4 strips)	445	252	1,055	903	393	47	254
MIC-PICS (JS, 8 strips)	605	347	785	549	442	90	369
MIC-C-WASM-PICS (8 strips)	1,123	584	1,791	1,625	1,173	252	676
HTJ2K (OpenJPH WASM)	287	129	779	705	127	9	111
JPEG-LS (CharLS WASM)	421	218	2,951	1,660	325	36	176
JPEG-XL*	391	98	6,256	6,256	98	6	98

*Informational row: native reference decode, not a browser decode; we did not identify a browser JPEG XL decoder capable of lossless 16-bit grayscale at the time of writing (Section II). Bold marks the fastest live browser decode per study among the parallel-capable rows; MIC-C-WASM single-thread is faster on the two 64×64 studies, where worker dispatch overhead exceeds the per-frame decode work.

TABLE XX

COMPRESSED PAYLOAD (TOTAL KB PER STUDY) FOR THE CINE CORPUS: CODEC-LEVEL SIZE COMPARISON. MIC SIZES ARE THE FOUR-STATE SINGLE-FRAME FILES; PICS STRIP CONTAINERS DIFFER BY AT MOST A FEW PERCENT EXCEPT WHERE NOTED.

Codec	Card. MR	XA	NM	Enh. MR	Enh. CT	DBT	CT ser.
MIC (4-state)	387	749	17	37	104	10,794	3,366
HTJ2K	435	701	16	39	126	11,282	3,053
JPEG-LS	367	619	12	36	78	10,192	2,929
JPEG-XL	165	571	10	34	54	8,813	2,691

TABLE XXI

MODELLED TIME-TO-DISPLAY (MS) FOR THE 7.18 MB CR RADIOGRAPH: ROUND-TRIP LATENCY + TRANSFER + MEASURED BROWSER DECODE (Eq. (1), PROFILES OF TABLE X). COMPRESSED SIZES: MIC 2.04 MB (3.69×), HTJ2K 2.00 MB (3.77×), JPEG-LS 1.90 MB (3.96×). BOLD MARKS THE FASTEST CONFIGURATION PER COLUMN.

Decoder	Gigabit LAN	Hospital LAN	Home broadband	Cellular (4G)	Congested / satellite
MIC-JS (4-state)	157	305	813	3,466	17,071
MIC-C-WASM	34	182	690	3,343	16,948
MIC-C-WASM-PICS-8	21	169	677	3,330	16,935
HTJ2K (OpenJPH WASM)	142	287	784	3,382	16,710
JPEG-LS (CharLS WASM)	86	224	699	3,174	15,888

fragments, with the standard image-pixel attributes carried over unchanged because MIC is lossless on native sample values. Formal adoption would proceed through a DICOM Change Proposal with a public bitstream specification and conformance test vectors; in the interim, two partners on a link can negotiate a private transfer syntax UID, which allows deployment inside a controlled environment before standardisation completes.

F. Future work

Beyond the corpus expansion and held-out validation above: extending the browser matrix to additional engines and low-powered endpoints; a NEON multi-state kernel for ARM64 servers; a full in-process JPEG XL ratio and speed table on the reference platforms (and revisiting its browser feasibility if a 16-bit-capable WebAssembly decoder appears, which Table XX suggests would win the slow-network regime); streaming and

TABLE XXII

CODIC SELECTION MATRIX. EACH CRITERION IS NORMALISED SO THE BEST CODEC ON THAT ROW SCORES 1.00. QUANTITATIVE ROWS ARE GEOMETRIC MEANS ACROSS THE 39-IMAGE SET. BROWSER COMPATIBILITY: SHIPPED PURPOSE-BUILT JS OR WASM DECODER USED IN PRODUCTION = 1.00; WASM PORT OF THE NATIVE LIBRARY USED IN CLINICAL VIEWERS = 0.85; NO BROWSER PATH = 0.0. DEPLOYMENT EASE: ZERO EXTERNAL RUNTIME DEPENDENCIES = 1.00; UBIQUITOUS SYSTEM DEPENDENCY = 0.90; C/C++ LIBRARY REQUIRING CMAKE BUILD AND LINK INTEGRATION = 0.70. WEIGHT COLUMNS w_P , w_A , w_B REFLECT TYPICAL PRIORITIES FOR (P) PACS VIEWER / LOW-LATENCY DECODE, (A) ARCHIVAL STORAGE, (B) BROWSER DELIVERY.

Criterion	w_P	w_A	w_B	MIC	HTJ2K	JPEG-LS
Compression ratio	0.20	0.50	0.10	0.88	0.89	1.00
Decompression throughput (ARM64)	0.30	0.10	0.15	1.00	0.57	0.34
Encoding throughput (ARM64)	0.05	0.10	0.05	1.00	0.40	0.25
AMD64 native	0.10	0.05	0.05	1.00	1.00	1.00
ARM64 native	0.10	0.05	0.05	1.00	1.00	1.00
Browser compatibility	0.05	0.00	0.40	1.00	0.85	0.85
Deployment ease	0.20	0.20	0.20	1.00	0.70	0.70
Weighted score (P) viewer	—	—	—	0.98	0.75	0.70
Weighted score (A) archival	—	—	—	0.94	0.78	0.80
Weighted score (B) browser	—	—	—	0.99	0.77	0.74

progressive strip decode so first pixels appear before the full object arrives; and the transfer-syntax standardisation path.

X. CONCLUSION

Real-time DICOM viewing in a web PACS imposes a joint constraint that archival codec comparisons do not capture: lossless reconstruction at native 10–16-bit depth, decode fast enough for 25–60 FPS multi-frame viewing, and a decoder that runs in whatever environment the endpoint provides, from a native gateway to a browser tab, with dependencies small enough to ship and audit. MIC addresses that constraint with a deliberately small pipeline (spatial prediction, 16-bit run-length encoding, large-alphabet FSE) plus two decode-side parallel mechanisms (multi-state chains and strip-parallel PICS), implemented as a family of interoperable decoders over one bitstream: pure Go, single-file C with SIMD, a ~20 KB dependency-free JavaScript module, and 20–30 KB WebAssembly builds with an optional pthreads pool. On a 39-image, eleven-modality corpus MIC reaches a 3.36× geometric-

mean ratio (within about 2% of HTJ2K, about 88% of JPEG-LS) and is the fastest of four in-process decoders on all 39 images on ARM64 and 36 of 39 on AMD64. Across the browser deployment matrix, identical bytes decode at ~51 MB/s in pure JavaScript, ~422 MB/s in C-WASM, and ~1.8 GB/s in threaded C-WASM on a 7.18 MB radiograph, spanning the 25–60 FPS target with margin on every study including tomosynthesis, while a Go-to-WASM build of the same codec demonstrates that managed-runtime WASM can be slower than tuned JavaScript. The time-to-display model separates the regimes where these decoder choices dominate (fast networks) from those where only ratio matters (slow networks). The individual stages are classical; the contribution is a codec-and-decoder-family design in which one archived object serves every deployment tier of a web PACS at interactive rates, verified bit-exact at each tier. MIC is open source: <https://github.com/pappuks/medical-image-codec>.

REFERENCES

- [1] NEMA, “Digital Imaging and Communications in Medicine (DICOM),” PS3.5—Data Structures and Encoding, <https://www.dicomstandard.org/>, 2024.
- [2] D. S. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Springer, 2002.
- [3] D. S. Taubman, “High throughput JPEG 2000 (HTJ2K): Algorithm, performance evaluation, and potential,” *Proc. SPIE*, vol. 11137, 2019.
- [4] D. Taubman *et al.*, “High Throughput JPEG 2000 for Video Content Production and Delivery Over IP Networks,” *Front. Signal Process.*, vol. 2, Article 885644, Apr. 2022.
- [5] M. J. Weinberger, G. Seroussi, and G. Sapiro, “The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS,” *IEEE Trans. Image Process.*, vol. 9, no. 8, pp. 1309–1324, 2000.
- [6] X. Wu and N. Memon, “Context-based, adaptive, lossless image coding,” *IEEE Trans. Commun.*, vol. 45, no. 4, pp. 437–444, 1997.
- [7] J. Sneyers and P. Wuille, “FLIF: Free Lossless Image Format based on MANIAC Compression,” *Proc. IEEE ICIP*, 2016.
- [8] B. W. Genereaux *et al.*, “DICOMweb™: Background and Application of the Web Standard for Medical Imaging,” *J. Digit. Imaging*, vol. 31, no. 3, pp. 321–326, 2018.
- [9] E. Ziegler *et al.*, “Open Health Imaging Foundation Viewer: An Extensible Open-Source Framework for Building Web-Based Imaging Applications to Support Cancer Research,” *JCO Clin. Cancer Inform.*, vol. 4, pp. 336–345, 2020.
- [10] A. Haas *et al.*, “Bringing the Web up to Speed with WebAssembly,” *Proc. 38th ACM SIGPLAN PLDI*, pp. 185–200, 2017.
- [11] A. Zakai, “Emscripten: an LLVM-to-JavaScript compiler,” *Proc. ACM OOPSLA Companion (SPLASH)*, pp. 301–312, 2011.
- [12] J. Alakuijala *et al.*, “JPEG XL next-generation image compression architecture and coding tools,” *Proc. SPIE 11137*, Sep. 2019.
- [13] F. Mentzer, E. Agustsson, M. Tschannen, R. Timofte, and L. Van Gool, “Practical Full Resolution Learned Lossless Image Compression,” *Proc. IEEE/CVF CVPR*, pp. 10629–10638, 2019.
- [14] S. Cao, C.-Y. Wu, and P. Krähenbühl, “Lossless Image Compression through Super-Resolution,” arXiv:2004.02872, 2020.
- [15] J. Duda, “Asymmetric numeral systems,” arXiv:0902.0271, 2009.
- [16] J. Duda, “Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding,” arXiv:1311.2540, 2013.
- [17] Y. Collet, “Finite State Entropy—A new breed of entropy coder,” <https://github.com/Cyan4973/FiniteStateEntropy>, 2013.
- [18] Y. Collet and M. Kucherawy, “Zstandard Compression and the ‘application/zstd’ Media Type,” RFC 8878, IETF, Feb. 2021.
- [19] S. Mahapatra and K. Singh, “An FPGA-based implementation of multi-alphabet arithmetic coding,” *IEEE Trans. Circuits Syst. I*, vol. 54, no. 8, pp. 1678–1686, 2007.
- [20] F. Giesen, “Interleaved entropy coders,” arXiv:1402.3392, 2014.
- [21] F. Lin, K. Arunrangsirilert, H. Sun, and J. Katto, “Recoil: Parallel rANS Decoding with Decoder-Adaptive Scalability,” *Proc. 52nd ICPP ’23*, pp. 31–40, 2023.
- [22] A. Weissenberger and B. Schmidt, “Massively Parallel ANS Decoding on GPUs,” *Proc. 48th ICPP ’19*, Article 100, 2019.
- [23] D. Le Gall and A. Tabatabai, “Sub-band coding of digital images using symmetric short kernel filters and arithmetic coding techniques,” *Proc. IEEE ICASSP*, pp. 761–764, 1988.
- [24] W3C, “Portable Network Graphics (PNG) Specification (Second Edition),” ISO/IEC 15948:2003, Nov. 2003.
- [25] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [26] F. Liu *et al.*, “The Current Role of Image Compression Standards in Medical Imaging,” *Information*, vol. 8, no. 4, p. 131, Oct. 2017.
- [27] D. A. Clunie, “Lossless Compression of Grayscale Medical Images: Effectiveness of Traditional and State-of-the-Art Approaches,” *Proc. SPIE Medical Imaging*, 2000.
- [28] A. N. Aous, “OpenJPH: An open-source implementation of High-Throughput JPEG 2000,” <https://github.com/aous72/OpenJPH>, 2024.
- [29] Team CharLS, “CharLS: A C/C++ JPEG-LS library implementation,” <https://github.com/team-charls/charls>, 2024.
- [30] NEMA Working Group 4 (WG-04), “DICOM Compression Sample Images,” National Electrical Manufacturers Association, <https://www.dicomstandard.org/Resources/Open-Content/Compression-Samples>, 2024.
- [31] D. A. Clunie, “Breast Tomosynthesis Case 1,” DICOM sample images, <http://www.dclunie.com/pixelmedimagescases/tomo/case1/>, 2009.
- [32] R. Geldreich, “Quick survey of the lossless decompression Pareto frontier,” 2015, <http://richg42.blogspot.com/2015/11/the-lossless-decompression-pareto.html>.
- [33] J. Sneyers, “JPEG XL and the Pareto front,” Cloudinary Blog, 2022, <https://cloudinary.com/blog/jpeg-xl-and-the-pareto-front>.
- [34] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. New York, NY, USA: McGraw-Hill, 1980.
- [35] K. Sayood, *Introduction to Data Compression*, 5th ed. Burlington, MA, USA: Morgan Kaufmann, 2017.
- [36] D. A. Clunie, “DICOM support for compression: More than JPEG,” *Medical Imaging Informatics and Teleradiology (MIIT)*, 2009, https://www.dclunie.com/papers/MIIT_2009_DicomCompression_Clunie.pdf.